

Rapport intermédiaire pour le “Projet de programmation”

Pour la date du 15 Mars, 2025

Cotrez Jules et Shay Ronen

15 mars 2025

1 Introduction

1.1 But du projet

Le but de notre projet est d’implémenter dans un langage fonctionnel (ici Ocaml) un jeu de plateformes en deux dimensions, dans lequel l’objectif est de réaliser des niveaux consécutifs en combattant des ennemis ainsi qu’en s’aidant d’items dispersés un peu partout dans le jeu.

1.2 Métrique

Nous considérerons notre projet comme réussi si nous parvenons à réaliser un jeu complet avec des déplacements (au minimum les déplacements classiques , un grappin qui permet un mouvement de balancier, et un jetpack), plusieurs niveaux, des ennemis et des items divers. Le reste (jeu en réseau, IA pour les ennemis, animation pour les attaques, ...) sera considéré comme du bonus.

2 Implémentation

2.1 réalisation du projet

Nous réalisons le projet en nous attribuant chacun une des tâches du planing (pas de branche séparée pour le moment). Nous utilisons le système de compilation Dune afin de gérer facilement l’organisation des fichiers et les dépendances. Pour l’interface graphique, nous nous servons de la librairie Raylib.

2.2 partie du logiciel codée

Pour le moment nous avons codé les déplacements du joueur (grappin inclus), l’affichage d’un menu ainsi que du premier niveau (avec plateformes et un ennemi), le parsing de fichiers Json afin de facilement modifier la description des

niveaux (nombre et position des plateformes et des ennemis). De plus nous avons veillé à implémenter la plus grande partie possible de manière fonctionnelle en évitant les effets de bord (c'est là toute la difficulté et l'intérêt de notre projet!). Les uniques effets de bords proviennent de l'utilisation de librairie Raylib pour l'affichage de l'interface graphique.

2.3 structure en modules/packages

Comme dit précédemment, nous avons utilisé le gestionnaire Dune pour instancier la structure de notre projet. Actuellement, nous avons deux classes principales séparées dans deux fichiers ml (+ deux fichiers mli associés pour l'interface) : jeu et joueur. La classe joueur contient tout ce qui est nécessaire à l'instanciation d'un nouveau joueur (joueur principal comme ennemi, nous ferons la distinction dans l'implémentation du jeu). Elle possède donc tous les attributs utiles tels que les dimensions, la direction, le vecteur de position, ... La classe jeu contient l'initialisation et la boucle principale du jeu. Elle possède également des structures facilitant la gestion des différentes entités (joueurs, ennemis, plateformes).